DESIGN AND APPLY CONTEXT AWARE ACCESS CONTROLS BASED ON USER IDENTITY

Group details

Name	College Name	Contact
SATHYA M	POLLACHI COLLEGE OF ARTS AND SCIENCE	8075650219 23nmcssathya@gmail.com
SANGEETHA M	POLLACHI COLLEGE OF ARTS AND SCIENCE	84388 72859 23nmcssangeetha@gmail.com
DHANUSHYA M	POLLACHI COLLEGE OF ARTS AND SCIENCE	78068 77027 23nmcsdhanushya@gmail.com
NIMSIYABEGM K	POLLACHI COLLEGE OF ARTS AND SCIENCE	95672 78398 23nmcsnimsiyabegam@gmail.com

Project Details

Title: DESIGN AND APPLY CONTEXT AWARE ACCESS CONTROLS BASED ON USER IDENTITY

Intro :

Context-aware access control is a security approach that grants or restricts access to resources based on a user's identity and contextual information. Instead of relying only on usernames and passwords, it considers factors such as location, device type, time, and user behavior. Designing and applying context-aware access controls helps organizations improve security by ensuring that only authorized users can access sensitive systems under appropriate conditions. This approach plays an important role in modern identity and access management systems.

Abstract:

Context-aware access control is an advanced security approach that enhances traditional access control mechanisms by considering both user identity and contextual information. This system evaluates factors such as user role, location, device type, and time of access before granting permission to resources. By analyzing these contextual elements, organizations can ensure that only authorized users access sensitive data and systems under appropriate conditions. Designing and applying context-aware access controls improves security, reduces unauthorized access, and supports flexible and dynamic access management in modern computing environments.

Use case scenario

1. **Office Network Access**An employee logs into the company system from the office using a registered device during working hours. The system verifies the user identity and context, then grants full access to company resources.

1. **Remote Access from Home**A user tries to access company data from home. The system checks the identity and detects a different location, so it requests additional authentication (such as OTP) before allowing access.

1. **Access from Unknown Device**When a user logs in from an unrecognized device, the system identifies the new device and limits access until the device is verified

1. **Access at Unusual Time**If a user attempts to log in late at night outside normal working hours, the system flags it as unusual and may restrict or monitor the access.

1. **Multiple Failed Login Attempts**If several incorrect login attempts occur, the system temporarily blocks the user account to prevent unauthorized access and protect sensitive data.
2.

Stage 1

Tilte: Owasp top 10

Overview:

OWASP Top 10 (2021) – Short Overview

1. **Broken Access Control** – Users access data or functions without proper permission.
2. **Cryptographic Failures** – Sensitive data not properly encrypted.
3. **Injection** – Malicious input executed as commands (e.g., SQL Injection).
4. **Insecure Design** – Security weaknesses due to poor application design.
5. **Security Misconfiguration** – Incorrect system or server settings.
6. **Vulnerable Components** – Using outdated libraries or frameworks.
7. **Authentication Failures** – Weak login or session management.
8. **Software & Data Integrity Failures** – Untrusted updates or code.
9. **Logging & Monitoring Failures** – Attacks not detected due to poor logging.
10. **Server-Side Request Forgery (SSRF)** – Server makes malicious internal requests.

S.no	Vulnarability name	CWE/CVE no.
1	**Absolute Path Traversal**	CWE 36
2	**Sensitive Cookie in HTTPS Session Without 'Secure' Attribute**	CWE 614
3	**Sensitive Cookie in HTTPS Session Without 'Secure' Attribute**	CWE 1104

Report:

Vulnerability name: **Absolute Path Traversal**

Cwe/cve number: CWE 36

Owasp category: **A01:2025 Broken Access Control**

Description: **Absolute Path Traversal** is a software weakness that occurs when an application uses external input to construct an absolute file path without adequate validation or normalization. Attackers can exploit this flaw to access files or directories outside the intended boundaries of the application, potentially exposing sensitive data or enabling code execution.

Buisness impact:

Common Weakness Enumeration
A community-developed list of SW & HW weaknesses that can become vulnerabilities

Top 25 | Top HW CWE | New to CWE? Start here!

CWE-36: Absolute Path Traversal

Weakness ID: 36
Vulnerability Mapping: ALLOWED
Abstraction: Base

View customized information: Conceptual | Operational | Mapping Friendly | Complete | Custom

Description

The product uses external input to construct a pathname that should be within a restricted directory, but it does not properly neutralize absolute path sequences such as "/abs/path" that can resolve to a location that is outside of that directory.

Extended Description

This allows attackers to traverse the file system to access files or directories that are outside of the restricted directory.

Common Consequences

Impact	Details
Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** The attacker may be able to create or overwrite critical files that are used to execute code, such as programs or libraries.
Modify Files or Directories	**Scope: Integrity** The attacker may be able to overwrite or create critical files, such as programs, libraries, or important data. If the targeted file is

Stage 2

Title: Nessus scan

Overview:

Nessus scans targets (servers, PCs, websites, or networks) and checks them against a large database of known vulnerabilities.

How a Nessus Scan Works

1. **Target Selection** – Define the IP address, domain, or network to scan.
2. **Scanning** – Nessus runs automated tests using plugins to detect vulnerabilities.
3. **Analysis** – The scanner analyzes services, ports, software versions, and configurations.
4. **Report Generation** – Results are categorized by severity (Critical, High, Medium, Low, Informational).

What Nessus Can Detect

- Open ports and exposed services
- Missing security patches
- Misconfigurations
- Known software vulnerabilities
- Weak SSL/TLS settings

S.no	Vulnerability name	Severity	Plugins
1.	Cross-site scripting (stored)	Low	Web application Scanner
2.	Broken Access Control	Medium	SSH weak HMAC Algorithms Supported
3.	SQL Injection	Medium	N/A
4.	Broken Authentication	Low	HSTS Missing from Server
5.	Insecure Direct Object Reference	Info	Service Detection
6.	Security Misconfiguration	Low	SSL cipher Block Chaining cipher suites
7.	Cross-Site Request Forgery	Low	HTTP Form- Based Authentication Detected
8.	Cleartext Transmission of Sensitive Information	Medium	Cleartext Transmission of Sensitive Information over HTTP
9.	ClickJacking	Info	HTML Rendering Layer Vulnerability
10.	webserver allows password auto-completion	Low	SSL/TLS Diffie-Hellman Modulus<=1024 bits(Logjam)

Report:

1. VULNERABILITY NAME: Cross site scripting (stored)

SEVERITY: Low

PLUGIN: web application scanner

PORT: N/A

DESCRIPTION: It occurs when a malicious script is injected directly into a vulnerable web application. Reflected XSS involves the reflection of a malicious script of a web application, onto a user's browser.

SOLUTION:

• When interacting with databases, use parameterized statements or prepared statements to avoid SQL injection, which can be a vector for XSS.

• Sanitize and validate all user inputs and ensure that any data displayed on the web page properly encoded. Use output encoding libraries or functions to prevent script injection.

BUSINESS IMPACT: The application stores dangerous data in a database, message forum, visitor log, or other trusted data store. Later, the dangerous data is subsequently read back into the application and included in dynamic content. From an attacker's perspective, the optimal place to inject malicious content is in an area that is displayed to either many users or particularly interesting users. Interesting users typically have elevated privileges in the application or interact with sensitive data that is valuable to the attacker. If one of these users executes malicious content, the attacker may be able to perform privileged operations on behalf of the user or gain access to sensitive data belonging to the user.

1. VULNERABILITY NAME: Broken access control

SEVERITY: medium

PLUGIN: SSH weak HMAC Algorithms supported

PORT: N/A

DESCRIPTION: When access control checks are not applied consistently - or not at all - users are able to access data or perform actions that they should not be allowed to perform. This can lead to a wide range of problems, including information exposures, denial of service, and arbitrary code execution.

SOLUTION:

• Ensure that the application has a robust authentication and authorization mechanism in place. Users should be authenticated before any sensitive actions are performed, and authorization checks should be conducted to verify that users have the necessary permissions for the requested operation.

• Ensure that session management is secure. Use strong session IDs, enforce session timeouts, and regenerate session tokens after login. Implement session fixation protection to prevent session hijacking

BUSINESS IMPACT: wide-ranging business impact, including financial losses, reputation damage, legal consequences, operational disruption, and loss of customer trust. Addressing this vulnerability is crucial to protect the organization and its stakeholders from these potential negative effects.

1. VULNERABILITY NAME: SQL injection

SEVERITY: medium

PLUGIN: N/A (typically would be a security testing tool)

PORT: port 80/443

DESCRIPTION: The product constructs all or part of an SQL command using externally influenced input from an upstream component, but it does not neutralize or incorrectly neutralizes special elements that could modify the intended SQL command when it is sent to a downstream component.

SOLUTION:

• If parameterized statements are not feasible, use proper input validation and escaping mechanisms to sanitize user inputs before they are used in SQL queries. This helps prevent malicious code injection.

• Utilize stored procedures for database operations. This can help separate SQL code from application code and reduce the risk of SQL injection.

• Implement strict input validation for all user inputs, ensuring that data adheres to the expected format and structure.

BUSINESS IMPACT: In summary, it is crucial to underscore that CWE-89, known as SQL Injection, can exert a profound and diverse business impact. This encompasses critical facets such as data breaches, financial setbacks, harm to reputation, legal ramifications, and operational turmoil. Hence, the imperative of preventing and remedying SQL injection vulnerabilities cannot be overstated, as it is indispensable for fortifying the security and continuity of an organization's applications and data.

1. VULNERABILITY NAME: Broken authentication

SEVERITY: Low

PLUGIN: HSTS Missing from HTTPS Server (plugin id:142960)

PORT: port 443

DESCRIPTION: The system's authorization functionality does not prevent one user from gaining access to another user's data or record by modifying the key value identifying the data.

SOLUTION:

• Implement strong authentication mechanisms, such as multi-factor authentication (MFA) or two-factor authentication (2FA), to ensure that only authorized users can access the system.

• Implement proper authorization controls to ensure that users have the necessary permissions to access specific functions or data. Use role-based access control (RBAC) to manage user privileges effectively.

• Deploy intrusion detection and prevention systems (IDS/IPS) to monitor and block suspicious activities related to authentication and authorization.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-285, it is imperative that organizations place a strong emphasis on bolstering their authentication and session management practices. This should encompass the adoption of multi-factor authentication, the secure storage of credentials, meticulous session handling, and a commitment to conducting routine security assessments and testing. The rectification of these vulnerabilities stands as a critical imperative, safeguarding sensitive data, user identities, and the organization's overarching security stature and reputation.

1. VULNERABILITY NAME: Insecure Direct object Reference

SEVERITY: Info

PLUGIN: Service Detection (plugin id:22964)

PORT: port 80/443

DESCRIPTION: Insecure Direct Object Reference (IDOR) is a vulnerability that arises when attackers can access or modify objects by manipulating identifiers used in a web application's URLs or parameters. It occurs due to missing access control checks, which fail to verify whether a user should be allowed to access specific data.

SOLUTION:

• Utilize parameterized queries or prepared statements when interacting with databases. This prevents user-controlled input from being executed as SQL code. • Implement a Content Security Policy to mitigate the risk of cross-site scripting (XSS) attacks, which can be used to execute unauthorized queries.

• Implement account lockout mechanisms that temporarily suspend accounts after a specified number of unauthorized access attempts to prevent brute force attacks.

BUSINESS IMPACT: To effectively mitigate the business impact of CWE-639, organizations must make it a top priority to fortify their access control and authorization mechanisms. This entails the implementation of robust security measures, the regular conduct of comprehensive security assessments, and the deployment of intrusion detection systems to promptly identify and counter unauthorized access attempts. These measures stand as critical in the defence of sensitive data, the overall security of systems, and the preservation of the organization's esteemed reputation

1. VULNERABILITY NAME: Security Misconfiguration

SEVERITY: low

PLUGIN: SSL cipher Block chaining cipher suites

PORT: port 443

DESCRIPTION: The product specifies permissions for a security-critical resource in a way that allows that resource to be read or modified by unintended actors.

SOLUTION:

• Implement a change management process to document and review all changes to system configurations. This helps prevent unauthorized or unintended alterations. • Follow security guidelines provided by software and hardware vendors to ensure secure configurations. • Regularly perform compliance checks against industry standards and regulations to ensure that configurations remain in compliance with security requirements.

BUSINESS IMPACT: To effectively lessen the business consequences stemming from CWE-732, organizations must place a strong emphasis on bolstering their configuration management practices. This involves the establishment of secure default settings, the routine conduct of comprehensive security evaluations, and the deployment of intrusion detection systems to promptly detect and address misconfigurations and vulnerabilities. These actions are of paramount importance in safeguarding sensitive data, fortifying system security, and preserving the organization's esteemed reputation.

1. VULNERABILITY NAME: Cross-Site Request Forgery

SEVERITY: Low

PLUGIN: HTTP Form-Based Authentication Detected

PORT: port 8080/443

DESCRIPTION: Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. With a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. If the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. If the victim is an administrative account, CSRF can compromise the entire web application.

SOLUTION:

• Implement Synchronizer Token Patterns to include unique, randomly generated tokens in each HTTP request and validate these tokens on the server side to verify the authenticity of the request.

• Utilize the Same Site attribute in cookies to restrict their usage to same-site requests only, preventing them from being sent along with cross-site requests and effectively mitigating CSRF attacks.

• Utilize framework-specific protections and security features to prevent CSRF attacks. Many modern web frameworks have built-in mechanisms and libraries for handling CSRF vulnerabilities effectively.

• Implement a Content Security Policy to restrict the sources from which various types of content can be loaded. By specifying the trusted sources of content, you can minimize the risk of malicious code execution and reduce the likelihood of successful CSRF attacks.

BUSINESS IMPACT: To effectively reduce the business repercussions of CWE-352, organizations must prioritize secure coding practices, including the incorporation of anti-CSRF tokens, while also conducting routine security assessments and testing to detect and rectify CSRF vulnerabilities. Furthermore, educating users on secure browsing practices is pivotal in preventing CSRF attacks. These actions are of paramount importance in the protection of data and in upholding the trust of customers and partners.

1. VULNERABILITY NAME: Cleartext Transmission of Sensitive Information

SEVERITY: Medium

PLUGIN: Cleartext Transmission of Sensitive Information over HTTP

PORT: port 443

DESCRIPTION: The product transmits sensitive or security-critical data in cleartext in a communication channel that can be sniffed by unauthorized actors. Many communication channels can be "sniffed" (monitored) by adversaries during data transmission. For example, in networking, packets can traverse many intermediary nodes from the source to the destination, whether across the internet, an internal network, the cloud, etc. Some actors might have privileged access to a network interface or any link along the channel, such as a router, but they might not be authorized to collect the underlying data. As a result, network traffic could be sniffed by adversaries, spilling security-critical data.

SOLUTION:

• Educate users and employees on secure data handling practices, such as recognizing secure websites (look for "https") and avoiding insecure Wi-Fi networks.

• Employ data masking or redaction techniques to replace sensitive data with placeholders or cryptographic representations during transmission.

• Implement encryption for data in transit. Use secure encryption protocols such as TLS/SSL to protect sensitive information during transmission.

• Use secure communication protocols for transmitting data, such as HTTPS for web applications, and ensure that the selected protocols are kept up to date.

BUSINESS IMPACT: To effectively reduce the business consequences associated with CWE-319, organizations must prioritize the adoption of secure data transmission practices. This includes the utilization of encryption and robust, secure protocols. The routine conduct of security assessments and testing is pivotal in pinpointing and remedying vulnerabilities linked to data transmission. Furthermore, the education of users on secure data handling practices plays a vital role in proactively preventing data exposure incidents. These actions are of paramount importance in the protection of sensitive data and in preserving the trust of both customers and partners.

1. VULNERABILITY NAME: Clickjacking

SEVERITY: Info

PLUGIN: HTML Rendering Layer Vulnerability

PORT: port 80/443

DESCRIPTION: The web application does not restrict or incorrectly restricts frame objects or UI layers that belong to another application or domain, which can lead to user confusion about which interface the user is interacting with.

SOLUTION:

• Set the X-Frame-Options HTTP response header to deny or same-origin to control how your site can be framed. This is supported by most modern browsers. • Utilize frame-busting JavaScript code in web applications to prevent the embedding of your site within malicious frame's. This code can disrupt clickjacking attempts.

• Implement a Content Security Policy to restrict which domains can embed your site in iframes. This can help prevent unauthorized framing.

• Implement additional security controls to prevent UI redress attacks, such as clickjacking, within your web application.

BUSINESS IMPACT: To effectively reduce the business consequences resulting from CWE-1021, organizations must prioritize the implementation of protective measures, such as frame-busting code.

Simultaneously, educating users on safe browsing practices plays a crucial role in preventing clickjacking incidents. Additionally, routine security assessments and testing are pivotal for identifying and mitigating vulnerabilities associated with clickjacking. These actions are of paramount importance in upholding user trust, ensuring data protection, and safeguarding the organization's reputation.

1. VULNERABILITY NAME: web server allows password auto-completion

SEVERITY: Low

PLUGIN: SSL/TLS Diffie-Hellman Modulus <= 1024 Bits (Logjam)

PORT: port 443

DESCRIPTION: Weaknesses in this category are related to the design and implementation of data confidentiality and integrity. Frequently these deal with the use of encoding techniques, encryption libraries, and hashing algorithms. The weaknesses in this category could lead to a degradation of the quality data if they are not addressed.

SOLUTION:

• Implement secure password storage mechanisms, such as using strong and salted cryptographic hashing algorithms like crypt or script.

• Implement robust encryption key management practices, including secure key storage, key rotation, and access controls.

• Utilize well-established and proven cryptographic libraries and algorithms for encryption and decryption.

• Use secure encryption protocols, like TLS/SSL, for data transmission over networks to protect data in transit

BUSINESS IMPACT: To effectively lessen the business consequences associated with CWE-310, organizations must prioritize the adoption of secure cryptographic practices. This entails ensuring proper password storage and robust encryption key management. Simultaneously, conducting routine security assessments and testing is pivotal in detecting and mitigating vulnerabilities related to cryptographic issues. Striving for compliance with pertinent data protection regulations and industry standards is equally essential. These actions are of paramount importance in the protection of sensitive data, the upholding of user trust, and the preservation of the organization's esteemed reputation.

Stage 3

Title: Design and apply context aware access controls based on user identity

Overview: Dynamic access control that grants or restricts access based on **user identity, device posture, location, and application risk**.

How It Works:

1. **Evaluate Context:** Collect info about the user, device, location, and app risk.
2. **Policy Check:** Compare against predefined access rules.
3. **Decision:** Grant, deny, or require extra verification (e.g., MFA).
4. **Continuous Monitoring:** Adjust access if context changes.

Benefits:

- Enhances security by adapting to risk.
- Improves user experience for low-risk scenarios.
- Supports compliance with sensitive data regulations.

Identity: Identity represents **who the user is** and is the primary factor in determining access. It goes beyond just a username—it includes attributes, roles, and authentication strength.

Device posture: Identity represents **who the user is** and is the primary factor in determining access. It goes beyond just a username—it includes attributes, roles, and authentication strength.

Application risk: Identity represents **who the user is** and is the primary factor in determining access. It goes beyond just a username—it includes attributes, roles, and authentication strength.

Context aware access: Context-Aware Access dynamically **grants or restricts access to resources based on real-time context**, rather than just static user credentials. It evaluates multiple factors to reduce risk and improve security.

Conclusion

Stage 1: The **OWASP Top 10** identifies the most critical security risks for web applications, highlighting vulnerabilities such as injection flaws, broken access controls, and security misconfigurations. It emphasizes that many risks stem from poor coding practices, outdated components, and weak authentication. By raising awareness and promoting secure design, proper access controls, patch management, and regular testing, the Top 10 helps organizations prioritize security efforts. Ultimately, it serves as a practical guide for preventing attacks and protecting the confidentiality, integrity, and availability of web applications.

Stage 2: A **Nessus scan** provides a comprehensive view of an organization's security posture by automatically identifying vulnerabilities, misconfigurations, and missing patches across systems, networks, and applications. The scan categorizes findings by severity, helping security teams prioritize remediation efforts efficiently. Regular Nessus scans enable proactive risk management, reduce the likelihood of exploitation, and support compliance with regulatory standards. Overall, Nessus is an essential tool for maintaining visibility into vulnerabilities and strengthening an organization's defenses before attackers can exploit weaknesses.

Stage 3: Designing and applying context-aware access controls allows organizations to **enforce dynamic, risk-based security** by evaluating not just who the user is, but also the device posture, location, and application risk. By continuously assessing these factors, access decisions can adapt in real time—granting seamless access for low-risk scenarios while enforcing stricter controls for high-risk situations. Implementing CAAC enhances security, reduces the likelihood of unauthorized access, improves user experience, and supports compliance with regulatory requirements. Ultimately, it shifts access management from static, one-size-fits-all policies to **intelligent, context-driven protection** of critical resources.

Future scope

Stage 1:

1. **Cloud and API Security:** Expanding focus on cloud-native applications, microservices, and APIs, which are increasingly targeted by attackers.

2. **AI and Automation Threats:** Addressing vulnerabilities that arise from AI-driven applications and automated processes.

3. **Supply Chain Security:** Highlighting risks from third-party components, libraries, and software dependencies.

Stage 2:

1. **Cloud and Hybrid Environments** – Enhanced scanning for cloud platforms, containers, and hybrid infrastructures.

2. **Automated and Continuous Scanning** – Integration into DevSecOps pipelines for real-time vulnerability detection during development and deployment.

3. **AI-Driven Threat Detection** – Using machine learning to identify previously unknown or zero-day vulnerabilities.

Stage 3:

1. **AI and Machine Learning Integration** – Using behavioral analytics to detect anomalies and automatically adjust access in real time.
2. **Zero-Trust Adoption** – Fully embedding CAAC into zero-trust frameworks, where every request is continuously evaluated based on context.
3. **IoT and Edge Device Security** – Applying context-aware policies to a growing range of connected devices beyond traditional endpoints

Topics explored:

Owasp top 10,

scanning,

vulnerability,

awareness identity

Tools explored:

Zscaler,

Nessus,

Owasp

hackthissite